



CS401 MPP

W4D1 Overview

Java Generics

Wholeness Statement

- Java generics facilitate stronger type-checking
 - Making it possible to catch potential casting errors at compile time (rather than runtime)
 - In many cases eliminates the need for downcasting.
- Generics also make it possible to support the most general possible API for methods that can be generalized.
 - We see this in simple methods like max and sort, and also in the Stream methods like filter and map.
- Generics involve type variables that can stand for any possible type;
 - in this sense they embody a universal quality. Yet, it is by virtue of this universal quality that we are able to specify particular types (instead of using a raw List, we can use List<T>, which allows us to specify a list of Strings – List<String> -- rather than a list of Objects, as we have to do with the raw List).
 - This shows how the lively presence of the universal sharpens and enhances the particulars of individual expressions. Likewise, contact with the universal level of intelligence sharpens and enhances individual traits.



CS401 MPP

Java Generics

Introduction

Generic Parameters

- Prior to jdk 1.5, a collection of any type consisted of a collection of Objects
 - Downcasting was needed to get the correct type back out of the collection

```
List words = new ArrayList();
words.add("Hello");
words.add(" world!");
String s = ((String)words.get(0)) + ((String)words.get(1));
System.out.print(s); //output: Hello world!
```

- In jdk 1.5, generic parameters were added to the declaration of collection classes, so that the above code could be rewritten as follows:

```
List<String> words = new ArrayList<String>();
words.add("Hello");
words.add(" world!");
String s = words.get(0) + words.get(1);
System.out.print(s); //output: Hello world!
```

Benefits of Generics

1. Stronger type checks at compile time

- A Java compiler applies strong type checking to generic code and issues errors if the code violates type safety
- Detecting errors at compile time is always preferable to discovering them at runtime
- Especially since, otherwise, the problem might not show up until the software has been released

```
List myList = new myList();  
myList.add("Tom");  
myList.add("Bob");  
.  
.  
.  
// no compiler check to prevent this  
Employee tom = (Employee)myList.get(0);
```

Example of poor type-checking

Benefits of Generics

2. Reduced Downcasting

- Downcasting is considered an “anti-pattern” in OO programming.
- Typically, downcasting should not be necessary (though there are plenty of exceptions to this rule)
- Finding the right subtype should be accomplished with late binding.

```
//Populate with Triangles and Rectangles
ClosedCurve[] closedCurves = . . .
if(closedCurves[0] instanceof Triangle) {
    print((Triangle)closedCurve[0].area());
}
else {
    print((Rectangle)closedCurve[0].area());
}
```

Example of bad downcasting

Benefits of Generics

3. Supports the most general possible API for methods that can be generalized

```
public static Integer max0(List<Integer> list) {  
    Integer max = list.get(0);  
    for (Integer i : list) {  
        if (i.compareTo(max) > 0) {  
            max = i;  
        }  
    }  
    return max;  
}
```

Get the max element in a list
Only works for List<Integer>

```
public static <T extends Comparable<T>> T max1(List<T> list) {  
    T max = list.get(0);  
    for (T i : list) {  
        if (i.compareTo(max) > 0) {  
            max = i;  
        }  
    }  
    return max;  
}
```

A much more generic approach
works with a list of anything that
implements Comparable

Terminology and Conventions

- The code below uses an ArrayList
 - In the Java Libraries it is called ArrayList<T>
 - ArrayList is a ***generic class***
 - T is called a ***type variable***, or ***type parameter***

```
List<String> words = new ArrayList<String>();  
words.add("Hello");  
words.add(" world!");  
String s = words.get(0) + words.get(1);  
System.out.print(s); //output: Hello world!
```


Terminology and Conventions

- The declaration:

```
List<String> words; // read: "List of String"
```

- Is called a ***generic type invocation***
- String is (in this context) a ***type argument***
- List<String> is a ***parameterized type***
- List with the type remove is called a ***raw type***

When raw types are used where a parametrized type is expected, the compiler issues a warning because the compile-time checks that can usually be done with parametrized types cannot be done with a raw type.

Note the difference between generic class and parameterized type and between type variable and type argument.

In each case, the first makes use of a type variable (like T) and the second provides a concrete type (like String)

Commonly Used Type Variables

Letter	What it signifies
E	Element (used extensively by the Java Collections Framework)
K	Key
N	Number
T	Type
V	Value
S,U,V	2 nd , 3 rd , 4 th types



CS401 MPP

Java Generics

Creating Generic Classes

Creating a Generic Class / Interface

- The SimplePair class declaration introduces type variables **K** and **V**
 - These can then be used in the body of the class as types of variables and method arguments and return types
 - The same principle applies when defining a generic interface
- The type variables may be realized as any Java object type
 - Even user-defined
 - But not as a primitive type

```
SimplePair<Integer, String> pair  
    = new SimplePair<>(10123, "Jim");  
Integer employeeId = pair.getKey();
```

Usage Example

```
public class SimplePair<K,V> {  
    private K key;  
    private V value;  
  
    public SimplePair(K key, V value) {  
        this.key = key;  
        this.value = value;  
    }  
  
    public K getKey() { return key; }  
    public V getValue() { return value; }  
}
```

Implementing a Generic Interface

- Two ways to implement the interface:

```
public interface Pair<K,V> {  
    public K getKey();  
    public V getValue();  
}
```

- Create a parameterized type implementation
 - Similar to **EmpComparator** implements **Comparator<Employee>**
- Create a generic class implementation
 - Similar to **ArrayList<T>** implements **List<T>**

Next slides will show these in code
See: [lesson11.lecture.generics.pairexamples](#) for full code

Parameterized Type Implementation

```
public class MyPair implements Pair<String, Integer> {  
    private String key;  
    private Integer value;  
  
    public MyPair(String key, Integer value) {  
        super();  
        this.key = key;  
        this.value = value;  
    }  
  
    @Override  
    public String getKey() {  
        return key;  
    }  
  
    @Override  
    public Integer getValue() {  
        return value;  
    }  
}
```

Generic Class Implementation

```
public class OrderedPair<K, V> implements Pair<K, V> {  
    private K key;  
    private V value;  
  
    public OrderedPair(K key, V value) {  
        this.key = key;  
        this.value = value;  
    }  
  
    public K getKey() {  
        return key;  
    }  
  
    public V getValue() {  
        return value;  
    }  
}
```

Extending a Generic Class

- The same applies for extending a generic class
 - Either Create a generic subclass
 - `public class MyList<T> extends ArrayList<T>{`
 - Or create a parameterized type subclass
 - `public class MyList extends ArrayList<String>{`



CS401 MPP

Java Generics

How Java Implements Generics

Type Erasure

- The compiler turns the following generic code

```
List<String> words = new ArrayList<String>();  
words.add("Hello");  
words.add(" world!");  
String s = words.get(0) + words.get(1);  
System.out.print(s); //output: Hello world!
```

- Into the following non-generic code

```
List words = new ArrayList();  
words.add("Hello");  
words.add(" world!");  
String s = ((String)words.get(0)) + ((String)words.get(1));  
System.out.print(s); //output: Hello world!
```

Type Erasure

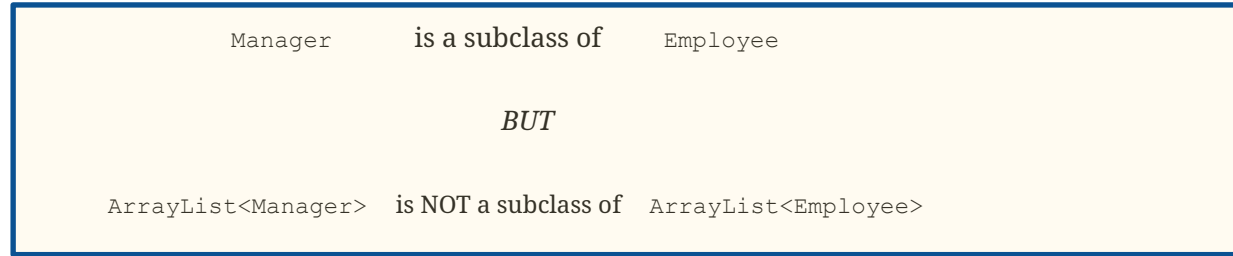
- Java is said to implement generics by *erasure* because
 - Parametrized types like **List<String>**, **List<Integer>** and **List<List<Integer>>**
 - Are all represented by the single type **List** at runtime
- Also *erasure* is the process of converting the first piece of code (on the previous slide) to the second
- The compiled code for generics will carry out the same downcasting as was required in pre-generics Java

Benefits of Type Erasure

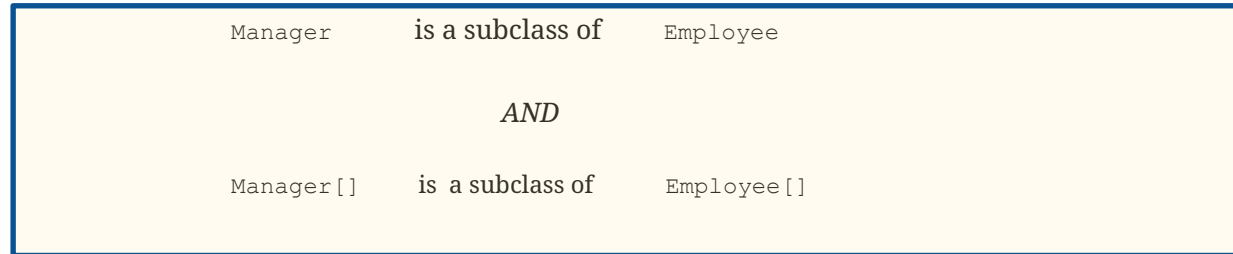
- No increase in the number of types in the language
 - in C++ each parametrized type makes a different type
- Backwards compatibility with non-generic code
 - For instance, in both generic and non-generic code, there is, at runtime, only one type List, so legacy code and generic code can intermingle without any difficulty

Unintuitive Aspects

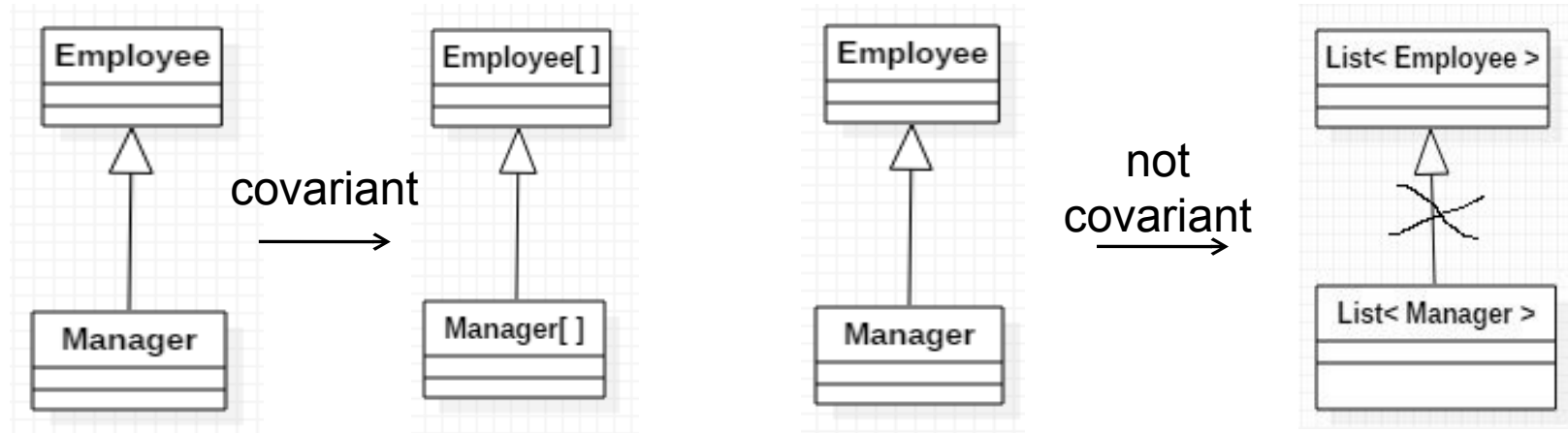
- Generic Subtyping is not Covariant



- Array Subtyping is Covariant



Array VS Generic Subtyping



Unintuitive Aspects

- Component type of an array is not allowed to be a type variable.
 - For example, we cannot create an array like this:

```
T[] arr = null; //this is ok so far
arr = new T[5]; //this produces a compiler error
```

The compiler has no information about what type of object to create

- This issue arises in Java's Collection classes:

```
class AbstractCollectionFirstTry {
    public static <T> T[] toArray(Collection<T> coll) {
        T[] arr = new T[coll.size()]; //compiler error
        int k = 0;
        for(T element : coll)
            arr[k++] = element;
        return arr;
    }
}
```

See: [lesson11.lecture.generics.toArray](#) on how to resolve this compiler error

Unintuitive Aspects

- The component type of an array is not allowed to a parameterized type
 - For example: you cannot create an array like this:

```
List<String>[] list = new List<String>[5];
```

- Another example:

```
class Another {  
    public static List<Integer>[] twoLists() {  
        List<Integer> list1 = Arrays.asList(1, 2, 3);  
        List<Integer> list2 = Arrays.asList(4, 5, 6);  
  
        // compiler error  
        return new List<Integer>[] { list1, list2 };  
    }  
}
```


Reifiable Types

- The problem on the previous slide exists because:
 - The component type of an array must be a reifiable type
- Reifiable Type Definition:
 - A type is reifiable if the type is completely represented at run time — when erasure does not remove any useful information.

Example

- The following situations show:

```
new String[size]
```

allocates an array, and stores in that array an indication that its components are of type String in other words: it reifies

```
new ArrayList<String>()
```

Allocates a list, but does not store in the list any indication of the type of its elements. In other words: it does not reify

```
new List<Integer>[] //not allowed
```

because the List type does not store component type information the resulting array is unable to store component type information which violates the rules for arrays

- Arrays need to have their full type information
 - Which parameterized types lose

This exercise illustrates one reason why generic subtyping is not allowed to be covariant

Exercise 11.1

- Examine the following code and answer 2 questions:
 1. In what step is the programmer attempting to use (generic subtype) covariance?
 2. Assuming the Java designers had decided to permit covariant generic subtyping: What would happen in the code that is undesirable and should not be allowed?
 - Hint: What is contained in the ints list at the end?

```
List<Integer> ints = new ArrayList<Integer>();  
ints.add(1);  
ints.add(2);  
List<Number> nums = ints;  
nums.add(3.14);  
System.out.print(ints);
```

Solution

```
List<Integer> ints = new ArrayList<Integer>();  
ints.add(1);  
ints.add(2);
```

assuming covariance, this step would be allowed

```
List<Number> nums = ints;
```

```
nums.add(3.14);  
System.out.print(ints);
```

output: [1, 2, 3.14] – not desirable



CS401 MPP

Java Generics

Generic Methods

Generic Methods

- Are methods that introduce their own type params.
 - This is similar to declaring a generic type
 - but the type parameter's scope is limited to the method where it is declared.
 - Static and non-static generic methods are allowed, as well as generic class constructors.

```
public static <K, V> boolean compare(SimplePair<K, V> p1, SimplePair<K, V> p2) {  
    return p1.getKey().equals(p2.getKey()) &&  
        p1.getValue().equals(p2.getValue());  
}
```

Calling a Generic Method

- Earlier versions of Java required that you specify the generic type arguments when calling a generic method

```
SimplePair<Integer, String> p1 = new SimplePair<>(1, "apple");  
SimplePair<Integer, String> p2 = new SimplePair<>(2, "pear");  
boolean areTheySame = Util.<Integer, String>compare(p1, p2);
```

- Current versions are always able to infer types

```
SimplePair<Integer, String> p1 = new SimplePair<>(1, "apple");  
SimplePair<Integer, String> p2 = new SimplePair<>(2, "pear");  
boolean areTheySame = Util.compare(p1, p2);
```

General Methods Generalize Behavior

- The following code counts occurrences of a target String inside a given input array
 - But the same procedure could be used to find a target of any given type in an array of the same type
 - Generic methods allow us to generalize from type String to an arbitrary type T

```
public static int countOccurrences(String[] arr, String target) {  
    int count = 0;  
    if (target == null) {  
        for (String item : arr)  
            if (item == null)  
                count++;  
    } else {  
        for (String item : arr)  
            if (target.equals(item))  
                count++;  
    }  
    return count;  
}
```


Another Example: find the max

1. Easy case:

- List of Integers

```
public static Integer max(List<Integer> list) {  
    Integer max = list.get(0);  
    for(Integer i : list) {  
        if(i.compareTo(max) > 0) {  
            max = i;  
        }  
    }  
    return max;  
}
```

2. Try to generalize to an arbitrary T

```
public static <T> T max1(List<T> list) {  
    T max = list.get(0);  
    for(T i : list) {  
        if(i.compareTo(max) > 0) {  
            max = i;  
        }  
    }  
    return max;  
}
```

Problem: T may not be a type that has a **compareTo** operation – we get a compiler error

Solution:

- It is possible to use the "extends" keyword to force the type T to be an implementer of the Comparable interface.
 - This produces a bounded type variable T extends Comparable

```
public static <T extends Comparable> T max(List<T> list) {  
    T max = list.get(0);  
    for(T i : list) {  
        if(i.compareTo(max) > 0) {  
            max = i;  
        }  
    }  
    return max;  
}
```

See: [lesson11.lecture.generics.genericprogrammingmax](#)

Generalizing Even Further

- The Comparable interface is also generic
 - For a class C, implementing Comparable implies:
 - that comparisons will be done between a current instance of C and an instance of possibly of some other type
 - The type argument used with Comparable is that type
 - For example we see on the String class:

```
String implements Comparable<String>
```

- Thus a String can be compared with another String

Generalizing Even Further

- This leads us to:

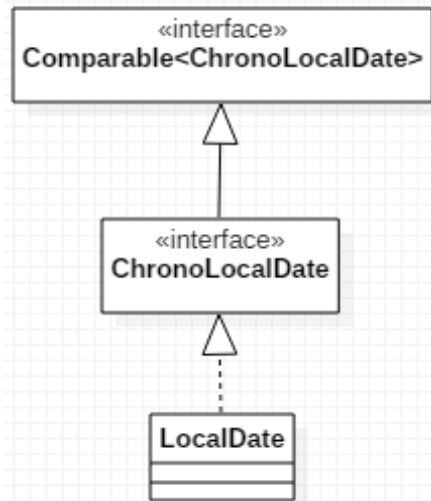
```
public static <T extends Comparable<T>> T max(List<T> list) {  
    T max = list.get(0);  
    for(T i : list) {  
        if(i.compareTo(max) > 0) {  
            max = i;  
        }  
    }  
    return max;  
}
```

This version of max can be used for most kinds of Lists, but there are exceptions

```
public static void main(String[] args) {  
    List<LocalDate> dates = new ArrayList<>();  
    dates.add(LocalDate.of(2011, 1, 1));  
    dates.add(LocalDate.of(2014, 2, 5));  
    LocalDate mostRecent = max(dates); //compiler error  
}
```

The Problem

- **LocalDate** does not implement **Comparable<LocalDate>**
 - Instead, the relationship to **Comparable** is the following:



What is needed is a max function that accepts types **T** that implement not just **Comparable<T>**, but even **Comparable<S>** for any supertype **S** of **T**.

Here, **T** is **LocalDate**. We want max to accept a list of **LocalDates** using a **Comparable<S>** for any supertype **S** of **LocalDate**.

The solution lies in the use of bounded wildcards.

LocalDate implements
Comparable<ChronoLocalDate>

Exercise 11.2

- The code for `countOccurrences` is in the package `lesson11.exercise_2` in the `InClassExercises` project.
 - Turn the method into a generic method so that it can be used to count occurrences of an object of any type in an array whose components are of the same type.
 - Then try writing the code for your generic method using a Stream pipeline instead of imperative code

Main Point 1

- Generic methods make it possible to create general-purpose methods in Java by declaring and using one or more type variables in the method.
 - This allows a user to make use of the method using any data type that is convenient, with full compiler support for type-checking.
- Likewise, when individual awareness has integrated into its daily functioning the universal value of transcendental consciousness, the awareness is maximally flexible, able to flow in whatever direction is required at the moment, free of rigidity and dominance of boundaries.



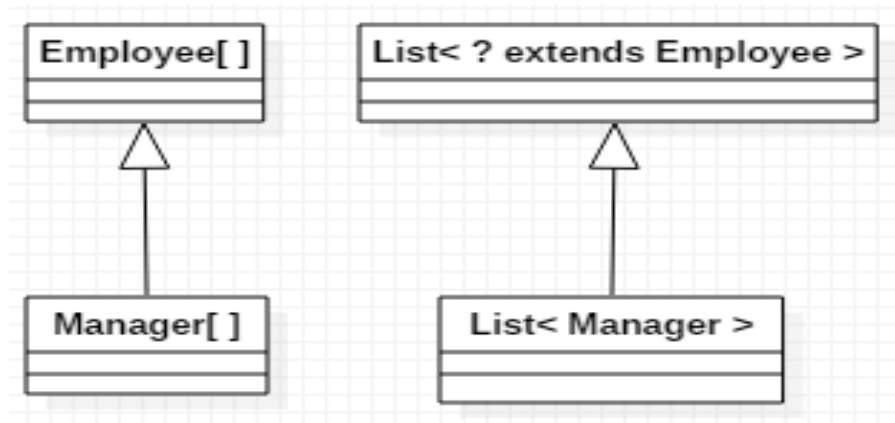
CS401 MPP

Java Generics

The ? extends Wildcard

The ? extends Bounded Wildcard

- The fact that generic subtyping is not covariant is inconvenient and unintuitive.
 - Eg. `List<Manager>` is not a subtype of `List<Employee>`
 - This is remedied to a large extent with the **extends** bounded wildcard.



The ? extends Bounded Wildcard

- The ? is a wildcard
 - The “bound” in **List<? extends Employee>** is the class **Employee**
 - **List<? extends Employee>** is called a parametrized type with a bound
- Usage:
 - If **C** is a subclass of **Employee**
 - **List<C>** is a subclass of **List<? extends Employee>**

Example

- While Manager extends Employee
 - This does not work

```
List<Manager> list1 = new ArrayList<>();  
List<Employee> list2 = list1; // compiler error
```

- But the following does work:

```
List<Manager> list1 = new ArrayList<>();  
List<? extends Employee> list2 = list1;
```

Applications of ? Extends

- The Java Collection interface has an addAll method:

```
interface Collection<E> {  
    ...  
    public boolean addAll(Collection<? extends E> c);  
    ...  
}
```

- The extends wildcard in the definition lets us:

```
List<Employee> list1 = ... // populate  
List<Manager> list2 = ... // populate  
list1.addAll(list2); // OK
```

Applications of ? Extends

- If the interface method had been declared like:

```
interface Collection<E> {  
    ...  
    public boolean addAllBad(Collection<E> c);  
    ...  
}
```

- Then for example **addAllBad** could only accept a **Collection of Employees**:

```
List<Employee> list1 = ... // populate  
List<Employee> list2 = ... // populate  
list1.addAll(list2); // OK
```

```
List<Employee> list1 = ... // populate  
List<Manager> list2 = ... // populate  
list1.addAll(list2); // Compiler Error!
```

Another Example Using addAll

- Since **Integer** and **Double** are both subtypes of **Number**
 - It follows that **List<Integer>** and **List<Double>** are subtypes of **List<? extends Number>**, and **addAll** maybe used on **nums** to add elements from both **ints** and **doubles**.

```
List<Number> nums = new ArrayList<Number>();  
List<Integer> ints = Arrays.asList(1, 2);  
List<Double> doubles = Arrays.asList(2.78, 3.14);  
nums.addAll(ints);  
nums.addAll(doubles);  
System.out.println(nums); //output: [1, 2, 2.78, 3.14]
```

Limitations of the extends Wildcard

- When the extends wildcard is used to define a parametrized type
 - Then the type cannot be used for adding elements
 - Example: with the **addAll** method we see:

```
List<Integer> ints = new ArrayList<Integer>();  
ints.add(1);  
ints.add(2);  
List<? extends Number> nums = ints;  
nums.add(3.14);           //compiler error  
System.out.println(ints.toString()); //output: [1, 2, 3.14]  
nums.add(null);           //OK
```

Limitations of the extends Wildcard

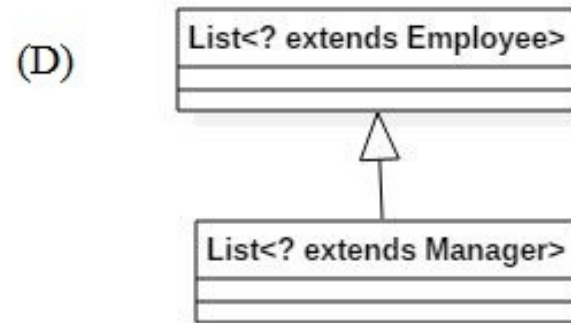
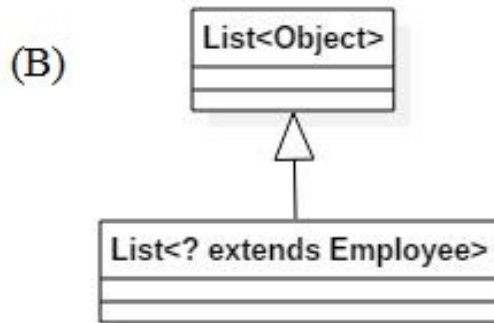
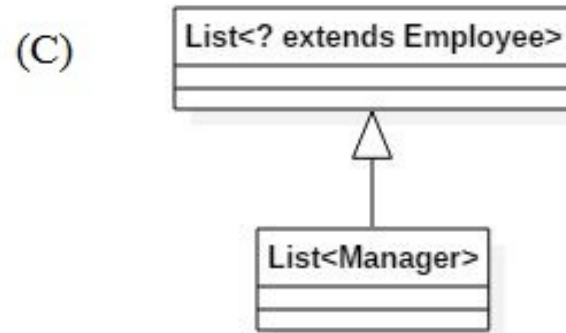
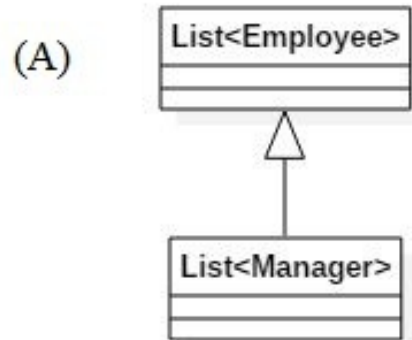
- The error arises because an attempt was made to insert a value in a parametrized type with extends wildcard parameter.
 - With the extends wildcard, values can be gotten but not inserted
- The difficulty is that adding a value to nums makes a commitment to a certain type (Double in this case)
 - Whereas nums is defined to be a List that accepts subtypes of Number, but which subtype is not determined.
 - The value 3.14 cannot be added because it might not be the right subtype of Number.

Although it is not possible to add to a list whose type is specified with the extends wildcard, this does not mean that such a list is read-only.

It is still possible to use:
remove, removeAll, retainAll
and the static methods from Collections:
sort, binarySearch, swap, shuffle

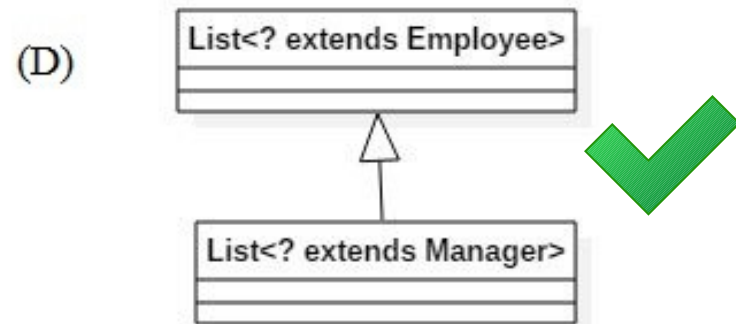
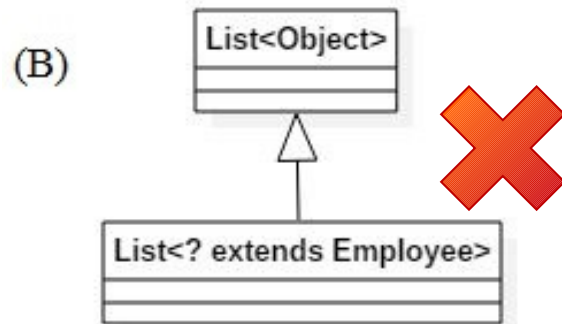
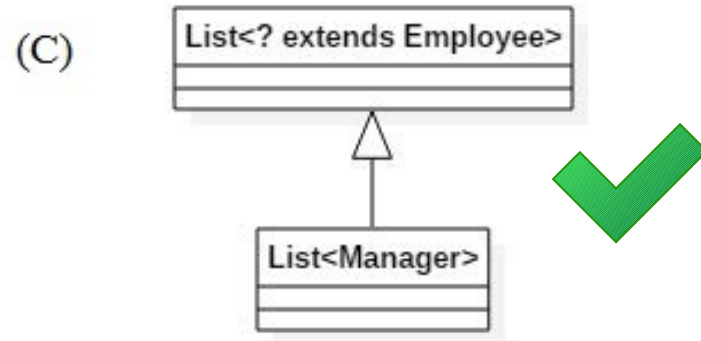
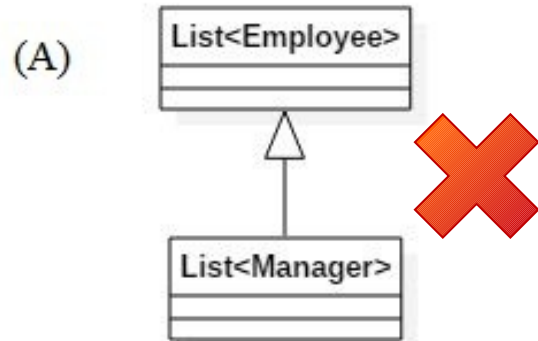
Exercise 11.3

- Determine which diagrams are correct



Solution

- Determine which diagrams are correct





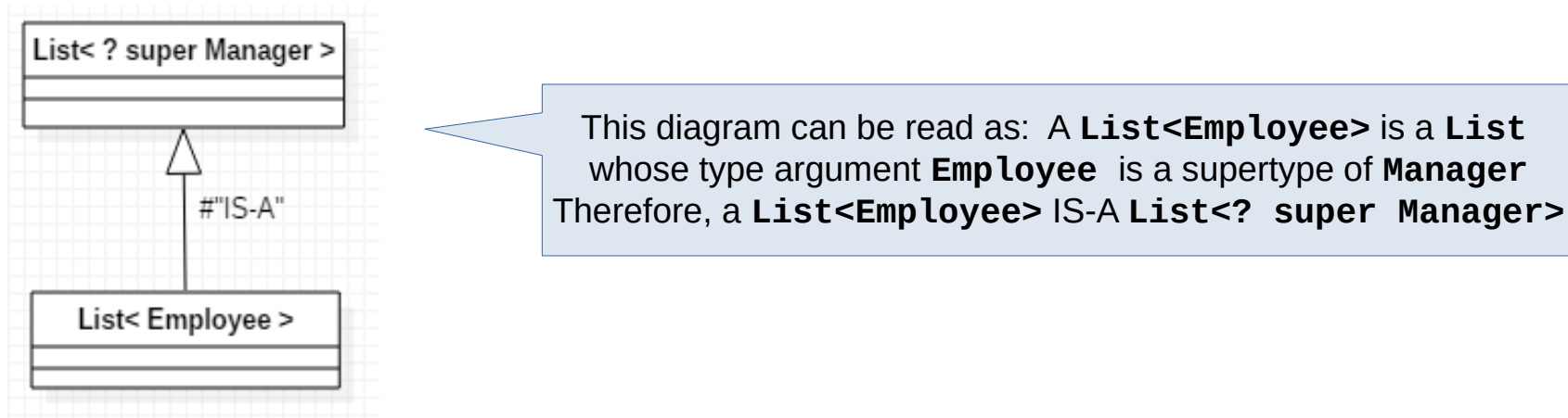
CS401 MPP

Java Generics

The ? super Wildcard

The ? super Bounded Wildcard

- The type **List<? super Manager>** consists of objects of any supertype of the **Manager** class
 - So objects of type **Employee** and **Object** are allowed



Limitations of the **super** Wildcard

- When the **super** wildcard is used to define a **Collection** of parametrized type
 - Elements can be gotten, but not typed
 - Therefore it is inconvenient to get elements

```
List<? super Integer> test = new ArrayList<>();  
test.add(5);  
System.out.println(test.get(0));
```

```
Integer val1 = test.get(0);    // compiler error  
Number val2 = test.get(0);    // compiler error  
Comparable val3 = test.get(0); // compiler error  
Object val4 = test.get(0);    // OK
```

Special case in which getting is possible
(You're getting an Object, which has .toString())

If we try to assign a return type other than Object
we get a compiler error. It has no way of knowing
which supertype of Integer is being gotten.

Get and Put Principle

- Use an **extends** wildcard when you only **get** values out of a structure
- Use a **super** wildcard when you only **put** values into a structure
- Don't use a wildcard at all when you both get and put values

Example 1

- This method takes a collection of numbers
 - Converts each to a double, and sums them up
 - The loop gets from an ? extends type of Collection

```
public static double sum(Collection<? extends Number> nums) {  
    double s = 0.0;  
    for(Number num: nums)  
        s += num.doubleValue();  
    return s;  
}
```

```
List<Integer> ints = Arrays.asList(1, 2, 3);  
Double val1 = sum(ints);    //output: 6.0
```

```
List<Double> doubles = Arrays.asList(2.78, 3.14);  
Double val2 = sum(doubles); //output: 5.92
```

No compiler errors because **List<Integer>**, **List<Double>** are subtypes of **Collection<? extends Number>**

Example 2

- We get from **source**
 - which is typed using **extends**
- we insert into **destination**
 - which is typed using **super**

It follows that any subtype of T may be gotten from source and any supertype of T may be inserted into destination

```
public static <T> void copy(List<? super T> destination, List<? extends T> source) {  
    for(int i = 0; i < source.size(); ++i) {  
        destination.set(i, source.get(i));  
    }  
}
```

```
List<Object> objs = Arrays.asList(2, 3.14, "four");  
List<Integer> ints = Arrays.asList(5, 6);  
//copy the narrow type (Integer) into the wider type (Object)  
Collections.copy(objs, ints);  
System.out.println(objs.toString());    //output: [5, 6, four]
```

Sample usage

Example 3

- Whenever you use the **add** method for a **Collection**
 - You are inserting values
 - So ? **super** should be used

```
public static void count(Collection<? super Integer> ints, int n) {  
    for (int i = 0; i < n; ++i) {  
        ints.add(i);  
    }  
}
```

The count method "counts" from 0 to n-1
adding these numbers to the input Collection ints.

Example 3 - Usage

- Since super was used we can:

```
List<Integer> ints1 = new ArrayList<>();  
count(ints1, 5);  
System.out.println(ints1); // output: [0,1,2,3,4]
```

```
List<Number> ints2 = new ArrayList<>();  
count(ints2, 5);  
ints2.add(5.0);  
System.out.println(ints2); // output: [0,1,2,3,4,5.0]
```

Count works because Number is a super class of Integer

```
List<Object> ints3 = new ArrayList<>();  
count(ints3, 5);  
ints3.add("five");  
System.out.println(ints3); // output: [0,1,2,3,4,"five"]
```

Count works because Object is a super class of Integer

The **add** methods shown here have nothing to do with the **<? super Integer>** on our **count** method
It's normal that you can add a Double to a list of numbers and a String to a list of Objects

Example 4

- Improving the **max** implementation

```
public static <T extends Comparable<T>> T max(List<T> list) {  
    T max = list.get(0);  
    for(T i : list) {  
        if(i.compareTo(max) > 0) {  
            max = i;  
        }  
    }  
    return max;  
}
```

We saw that this implementation was not general enough

```
public static void main(String[] args) {  
    List<LocalDate> dates = new ArrayList<>();  
    dates.add(LocalDate.of(2011, 1, 1));  
    dates.add(LocalDate.of(2014, 2, 5));  
    LocalDate mostRecent = max(dates); //compiler error  
}
```

It encounters a compiler error here

Example 4

- To ensure that the type **T** extends **Comparable<S>** for any supertype of **T**, we can use **? super**

```
public static <T extends Comparable<? super T>> T max(List<T> list) {  
    T max = list.get(0);  
    for(T i : list) {  
        if(i.compareTo(max) > 0) {  
            max = i;  
        }  
    }  
    return max;  
}
```

Using this version
eliminates the
compiler error

Need both Put and Get?

- Whenever you both put into and get out of the same structure
 - You should not use a wildcard

```
public static double sumCount(Collection<Number> nums, int n) {  
    count(nums, n);  
    return sum(nums);  
}
```

- The collection is passed to both sum and count
 - So its element type must both extend Number (because of sum)
 - And be a superclass of Integer (because of the count method)
 - The only classes that satisfy both are Number and Integer
 - In this code, Number was chosen

```
List<Number> nums = new ArrayList<Number>();  
double sum = sumCount(nums, 5); //sum is 10
```

Two Exceptions to The Get and Put Rule

1. In a Collection that uses the extends wildcard

- Null can always be added legally
- Null is the “ultimate” subtype

```
List<Integer> ints = new ArrayList<>();  
ints.add(1);  
ints.add(2);  
List<? extends Number> nums = ints;  
nums.add(null); //OK  
System.out.println(nums.toString()); //output: [1, 2, null]
```

Two Exceptions to The Get and Put Rule

2. In a Collection that uses the super wildcard

- Any object of type Object can be read legally
- Object is the “ultimate” supertype

```
List<? super Integer> list = new ArrayList<>();  
list.add(1);  
list.add(2);  
Object ob = list.get(0);  
System.out.println(ob.toString()); //output: 1
```

Main Point 2

- The Get and Put Rule describes conditions under which a parametrized type should be used only for reading elements (when using a list of type ? extends T)
- Other conditions under which the parametrized type should be used only for inserting elements (when using a list of type ? super T)
- And still other conditions under which the parametrized type can do both (when no wildcard is used)
- The Get and Put principle brings to light the fundamental dynamics of existence:
 - There is dynamism (corresponding to Put); there is silence (corresponding to Get) and there is wholeness, which unifies these two opposing natures (corresponding to Both)



CS401 MPP

Java Generics

Unbounded Wildcard

Unbounded Wildcard

- The wildcard **?** without the **super** or **extends** qualifier
 - Is called the *unbounded wildcard*
- **Collection<?>** is an abbreviation for
 - **Collection<? extends Object>**
- **Collection<?>** is the supertype of all parametrized type Collections

Wildcard Capture

- *Wildcard capture* is an important application of the unbounded wildcard
- Example: Say we want to copy the 0th element of a general list to the end of the list
 - Our first try might be:

```
public void copyFirstToEnd(List<?> items) {  
    items.add(items.get(0)); // compiler error  
}
```

Compiler error arises because we are trying to add to a List whose type involves the extends wildcard

Solution

- Write a helper method that *captures* the wildcard

```
public void copyFirstToEnd2(List<?> items) {  
    copyFirstToEndHelper(items);  
}  
  
private <T> void copyFirstToEndHelper(List<T> items) {  
    T item = items.get(0);  
    items.add(item);  
}
```

Passing items into the helper method causes the unknown type ? to be “captured” as the type T

In the helper method, getting and setting values is legal because we are not dealing with wildcards in that method

Exercise 11.4

- The following code attempts to reverse the elements of a generic list, but the code does not compile
 - Fix the code by creating a helper method that captures the wildcard.
 - Test your code with the main method that has been provided.
 - Code is in lesson11.exercise_4 of InClassExercises

```
public static void reverse(List<?> list) {  
    List<Object> tmp = new ArrayList<Object>(list);  
    for (int i = 0; i < list.size(); i++) {  
        list.set(i, tmp.get(list.size()-i-1));  
    }  
}
```



CS401 MPP

Java Generics

Closing Remarks

Generic Programming Using Generics

- Generic programming is the technique of implementing a procedure so that it can accommodate the broadest possible range of inputs.
- For instance, we have considered several implementations of a max function. The goal of generic programming in this case is to provide the most general possible max implementation.
- See demos `lecture.generics.max.BoundedTypeVariable` and `lecture.generics.max.BoundedTypeVariable2` for a development of examples leading to the most general possible version.

Connecting the Parts of Knowledge With the Wholeness of Knowledge

1. Using the raw Lists of pre-Java 1.5, one can accomplish the generic programming task of swapping two elements in an arbitrary list using the signature `void swap(List, int pos1, int pos2)`. Using this swap method requires the programmer to recall the component types of the List, and there are no type checks by the compiler.
2. Using generic Lists of Java 1.5 and the technique of wildcard capture, it is possible to swap elements of an arbitrary List with compiler support for type-checking, using the following signature:

```
<T> void swap(List<?> list, int pos1, int pos2)
```

3. **Transcendental Consciousness** is the universal value of the field of consciousness present at every point in creation.
4. **Impulses Within the Transcendental Field.** The presence of the transcendental level of consciousness within every point of existence makes individual expressions in the manifest field as rich, unique, and diversified as possible.
5. **Wholeness Moving Within Itself.** In Unity Consciousness, life is appreciated in the fullest possible way because the source of both unity and diversity have become a living reality.